NitroDrp Handbook

Running, updating and fixing your private file-transfer app

1. The big picture

NitroDrp is your own WeTransfer. People open a link, you upload big files, they download them. Nothing here is complicated once you know **which place does what**.

Place	What lives there	You touch it when
Your Ubuntu (WSL) machine	The code, Terraform, the AWS CLI, your admin AWS login, `terraform.tfvars`	Changing code, changing AWS settings
The Lightsail server (`ubuntu@ip-172-26-13-190`)	The running app in Docker, the secret `.env` file, the CloudFront signing key in `secrets/`	Updating the live site, changing a setting, reading logs
GitHub (`FunmbiO/NitroDrp`)	The code and its history. The only thing the two machines share	Merging changes
AWS	S3 (files), DynamoDB (link info), CloudFront (fast downloads), SES (email), IAM (permissions)	Almost never by hand. Terraform manages it
Namecheap	DNS records for jadedmedia.ca	Adding or fixing a record
Vercel	Your main site, and the browser-only demo at demo.jadedmedia.ca	Rarely. It redeploys itself

Your two sites:

- **drop.jadedmedia.ca** is the real app. Files go to your AWS account.
- **demo.jadedmedia.ca** is a harmless copy that runs entirely in the visitor's browser. Nothing is uploaded. It's for your portfolio.

How a transfer flows

1. You open drop.jadedmedia.ca, enter the access key, pick files and press **Create link**.
2. Your browser sends the files **straight to S3** in pieces. The server only signs the permission slips.
3. The link is ready. Copy it, or tick "Also send the link by email" and SES sends it for you.
4. The recipient opens the link and downloads through **CloudFront**. (The "Download all as .zip" button streams through your server instead.)
5. At expiry the link stops working at once, and AWS deletes the data on its own.

2. Which machine runs what

Rule of thumb: change things on your Ubuntu machine, run things on the server. The server never edits code. It only pulls it from GitHub and runs it.

Job	Where	Why
Edit code, run tests	**Local**	Your editor lives there
`terraform plan` / `apply`	**Local**	Terraform isn't installed on the server, and the server's AWS key is deliberately too weak to change infrastructure
`git push`, merging	**Local** + GitHub	
`git pull`, `docker compose up`	**Server**	That's where the app runs
Edit `.env`	**Server**	Secrets live only there
Read logs, restart the app	**Server**	
Create AWS access keys, request SES access	**Browser** (AWS console)	
Add or fix DNS records	**Browser** (Namecheap)	

Two files exist in only one place. Back both up (a password manager is ideal):

- `.env` on the server: your API key, AWS keys and settings.
- `infra/terraform/terraform.tfvars` and `backend.hcl` on your local machine. (Terraform's *state* is safely in S3.)

3. The routine for changing the app

```
local:   edit  ->  test  ->  commit  ->  push a branch
GitHub:  open a pull request  ->  merge  ->  delete the branch
server:  git pull  ->  rebuild
```

1. **Local:** make a branch, change things, run the tests (section 4).
2. **Local:** `git push -u origin your-branch`, then merge it on GitHub (Pull requests, Merge).
3. **Server:** pull and rebuild:

```
cd ~/NitroDrp
git checkout main && git pull
docker compose --profile public up -d --build
```

That's the whole deployment. The site is down for a few seconds while the API restarts. The demo on Vercel redeploys itself when `main` changes.

4. Commands you'll actually use

On the server

Command	What it does
`cd ~/NitroDrp`	Go to the app (do this first)
`docker compose --profile public ps`	Is everything running? You want `caddy`, `web`, `api` all "Up"
`docker compose --profile public logs api --tail 50`	The last 50 log lines from the API. **First stop when something is wrong**
`docker compose --profile public logs -f api`	Watch the log live (Ctrl+C to stop)
`docker compose --profile public logs caddy --tail 50`	HTTPS and certificate problems
`git checkout main && git pull`	Get the latest code
`docker compose --profile public up -d --build`	Rebuild and restart with the latest code **and** the current `.env`
`docker compose --profile public up -d`	Apply a changed `.env` without rebuilding
`docker compose --profile public down`	Stop everything
`nano .env`	Edit settings. Save with Ctrl+O, Enter, then Ctrl+X
`curl -s https://drop.jadedmedia.ca/api/health`	Quick "is it alive?" check
`docker system prune -f`	Free disk space from old images (safe)

> **Gotcha:** `docker compose restart` does *not* re-read `.env`. After editing `.env`, always run `docker compose --profile public up -d`.

On your local Ubuntu

Command	What it does
`cd ~/NitroDrp && git checkout main && git pull`	Start from the latest code
`git checkout -b short-name`	Start a new branch for a change
`git add -A && git commit -m "short message"`	Save your work (make commits small and short)
`git push -u origin short-name`	Send the branch to GitHub
`code .`	Open the project in VS Code (section 5)

Run the tests (do this before every push):

Command	Checks
`cd api && dotnet test`	The API: all the rules, limits, email, passwords, expiry
`cd web && npm test`	The web app and the demo backend
`cd web && npx tsc --noEmit`	Type errors in the web code
`cd infra/terraform && terraform test`	The AWS setup, offline, nothing real is touched
`bash dev/test-setup-env.sh`	The `.env` setup script
`python3 dev/check-links.py`	Broken links in the docs

Look at the app on your own machine:

Command	What you get
`cd web && npm install && npm run mock` (terminal 1), `npm run dev` (terminal 2)	The full UI at http://localhost:5173 with fake data. No AWS, no Docker. The quickest way to see a design change
`docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build`	The real app with fake S3, DynamoDB and email, at http://localhost:8080 (access key `dev-access-key-change-me`)
`python3 dev/smoke.py`	An end-to-end test of that stack: upload, resume, download, zip, email

AWS and Terraform (local only)

Command	What it does
`aws sts get-caller-identity`	Which AWS account am I logged in as? Run this first if Terraform acts strange

Command	What it does
`cd infra/terraform`	Go to the infrastructure folder
`terraform plan`	**Preview** changes. Changes nothing. Always run this first
`terraform apply`	Make the changes (type `yes`). Read the plan summary before you say yes
`terraform output`	Show the values you copy into the server's `.env`
`aws sesv2 get-email-identity --email-identity jadedmedia.ca --region us-east-2`	Is the email domain verified?

5. Your editor: VS Code

Use **Visual Studio Code**. It's free, it works with both the C# API and the React web app, and it connects to Ubuntu on Windows (WSL) so you edit the same files you run.

Setup (once):

1. Install VS Code on Windows, then the **WSL** extension.
2. In your Ubuntu terminal: `cd ~/NitroDrp && code .` The window's bottom-left corner shows "WSL: Ubuntu".
3. Install these extensions (VS Code offers them when you open the right files):
   - **C# Dev Kit** (the API; needs the .NET 10 SDK in Ubuntu: check with `dotnet --version`)
   - **HashiCorp Terraform** (the `.tf` files)
   - **Docker** (see containers and logs in a sidebar)
   - **GitLens** (optional, shows who changed what and when)

Looking at code: the map is in `docs/DEVELOPMENT.md`. The three places you'll visit most:

- `api/src/NitroDrp.Api/Services/ShareService.cs`: all the rules (limits, expiry, passwords, email)
- `web/src/pages/`: the screens (`UploadPage`, `CreatedPage`, `DownloadPage`)
- `infra/terraform/`: everything in AWS

Debugging, the easy way:

- **API:** open the **Testing** panel (the flask icon) and press the bug icon beside any test. VS Code stops on your breakpoints. Almost every rule has a test, so this is the quickest way to watch the code run.
- **Web app:** run `npm run mock` and `npm run dev`, open http://localhost:5173 in Chrome and press **F12**. The Console tab shows errors and the Network tab shows each request. VS Code can also attach: **Run, Start Debugging, JavaScript Debug Terminal**.
- **The live site:** you rarely debug it directly. Read `docker compose logs api` on the server instead.

6. Common jobs

Update the live site after merging a change: section 3.

Change a setting (the sender name, an email address, and so on): on the server, `nano .env`, then `docker compose --profile public up -d`.

Rotate the AWS access key (do this if a key is ever exposed):

1. AWS console, IAM, Users, `nitrodrp-api`, Security credentials, **Create access key**.
2. On the server, put the new id and secret in `.env` (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`).
3. `docker compose --profile public up -d`, then open the site and make a test link.
4. Back in the console, set the old key to **Inactive**, wait a day, then delete it.

Change AWS settings (expiry choices, turning email or CloudFront on or off): on your local machine edit `infra/terraform/terraform.tfvars`, run `terraform plan`, read it, then `terraform apply`. If it produced new values (`terraform output`), copy them into the server's `.env` and run `up -d`.

Add a DNS record: Namecheap, Domain List, Manage, Advanced DNS. In **Host** type only the part *before* `.jadedmedia.ca`. Namecheap adds the rest. Typing the whole name is the classic mistake.

Check email is healthy: run the `aws sesv2 get-email-identity` command above. You want `VerifiedForSendingStatus` true. If recipients show "Not sent", the usual cause is that SES is still in its sandbox (it only sends to addresses you verified) until AWS approves production access.

Is it using CloudFront? Make a link, press F12, open the Network tab, click a single file's download. The URL starts with `d7m58svsya041.cloudfront.net`. The zip button always goes through the server.

7. When something breaks

Symptom	Likely cause and fix
Site won't load at all	Server: `docker compose --profile public ps`. If anything is down, run `up -d`, then read `logs caddy`
Certificate or HTTPS error	The `drop` DNS record is missing or wrong (should be the Lightsail static IP). Also `logs caddy`
"Wrong access key"	The key you typed doesn't match `API_KEY` in the server's `.env`
Upload fails or stalls	Press **Resume upload** (progress is saved). Then `logs api`. Check the S3 bucket and key are right in `.env`

Symptom	Likely cause and fix
API won't start after an edit	`logs api` names the problem, usually a typo in `.env`. Email on needs `EMAIL_FROM_ADDRESS` and `PUBLIC_BASE_URL`
Email says "Not sent"	SES sandbox or an unverified domain. See the email check above and `docs/EMAIL.md`
Downloads fail on a new link	CloudFront settings in `.env` empty or wrong, or the key file in `secrets/` isn't readable by the app
`terraform` shows changes you didn't make	Run `aws sts get-caller-identity` first. Never apply a plan you don't understand. Ask before you say `yes`
A DNS record vanished	Re-add it from the list in `docs/DEPLOY.md` and `docs/LIGHTSAIL.md`. Keep a screenshot of your Advanced DNS page
Disk full on the server	`docker system prune -f`
Unsure what changed	`git log --oneline -10` on either machine

8. Secrets and safety

- **Never paste** your AWS **secret** access key, the `API_KEY`, or any `.pem` private key into chat, email or GitHub. Key *ids* (they start with `AKIA`) and CloudFront ids are not secret on their own.
- The files `.env`, `secrets/`, `terraform.tfvars` and `backend.hcl` are in `.gitignore` so they can't be committed by accident. Keep it that way.
- The server's AWS key can only touch this app's bucket, table, CloudFront and email domain. Your admin login can do everything, which is why Terraform runs only on your machine.
- Links are unguessable, optionally password protected, and expire. The access key is the only thing stopping a stranger from *creating* links, so treat it like a password.
- Emails never contain the password. If you protect a link, send the password through a different channel.

9. What it costs and its limits

Thing	Typical
Lightsail server	Roughly US$8 to 12 a month all in (the main cost)
S3, DynamoDB	Pennies at personal use. Files vanish at expiry

Thing	Typical
CloudFront	Free allowance covers light use
Email (SES)	About US$0.10 per 1,000 emails
Alert	A budget alert emails you if spend passes the limit set in Terraform

Limits you can change: expiry choices (1, 7, 30 days), up to 10 email recipients per send and 20 per link, rate limits per visitor.

10. Where to read more

File	For
`README.md`	The one-page summary
`docs/DEPLOY.md`	Setting up AWS and the server, and CloudFront
`docs/LIGHTSAIL.md`	The server, step by step
`docs/EMAIL.md`	Email, DNS records and leaving the SES sandbox
`docs/TERRAFORM_STATE.md`	Where Terraform keeps its memory, and recovery
`docs/DEMO.md`	The demo and embedding it in your portfolio
`docs/ARCHITECTURE.md`	How it's built and why
`docs/DEVELOPMENT.md`	Running and testing it

Golden rules: run the tests before you push, run `terraform plan` before `terraform apply`, and read the logs before you guess.